

General Features of Algorithms

Common features of definitions of Algorithms:
finite elementarity of the procedure in terms of elementary operations

deterministic $\Rightarrow$ next step is uniquely determined if there is one
procedure may not terminate on some input data, but we can recognise when it does terminate and **what the result** will be

We can think of **algorithms as Computable Functions** (e.g. RM-computable, TM-computable, etc.)

Partial Function: $f: X \rightarrow Y$ is such that:
 $X \rightarrow Y$ is the set of **partial functions** from X to Y
 $f(x) = y$ means $(x, y) \in f$
 $f(x) = y \wedge f(x) = y' \Rightarrow y = y'$
 $f(x)$ means $\exists y \forall y' (f(x, y) \wedge \neg f(x, y'))$
 f is **Total** if $\forall x \in X$ we have $f(x)$
 $X \rightarrow Y$ is set of **total functions** from X to Y

General Halting Problem is the decision problem with the set S of all pairs (A, D) where A is an **algorithm** and D is some **input datum** on which A is designed to operate
 $A(D) \downarrow$ holds for $(A, D) \in S$ if A applied to D eventually halts
The **Halting Problem** is **unsolvable (undecidable)**, i.e. no algorithm $H: S \rightarrow \{0, 1\}$ exists s.t. $H(A, D) = \begin{cases} 1 & \text{if } A(D) \downarrow \\ 0 & \text{if } A(D) \uparrow \end{cases}$

Characteristic Function $\chi_S: N \rightarrow \{0, 1\} \Rightarrow$
 $\chi_S(x) = \begin{cases} 1 & x \in S \\ 0 & x \notin S \end{cases}$

Register Machine Definitions

A Register Machine (RM) is specified by:
finite **registers** $R_0, \dots, R_n$ each can store one nat. N
a **program** $\Rightarrow$ finite list of **instructions**
each instruction of the form $L_i: B_i$
 B_i is **label** of $i+1$ th instruction, for $i=0, 1, \dots$
 B_i is **instruction body**, and is one of:
 $R^+ \rightarrow L^+$ $\Rightarrow$ add 1 to reg. R & jump to **inst.** L^+
 $R^- \leftarrow L^+$ $\Rightarrow$ subtract 1 from reg. R & jump to **inst.** L^+
else jump to L^+
 $HALT$ $\Rightarrow$ stop executing instructions
initial/final reg. contents related by **partial function**

RM Configuration $c = (r, r_0, \dots, r_n)$
 r = current label r_i = content of registers R_i
 $R_i = x$ [in config. c] means $c = (r, r_0, \dots, r_n)$ with $r_i = x$
initial configuration $\Rightarrow c_0 = (0, r_0, \dots, r_n)$

RM Computation $\Rightarrow$ finite (or infinite) sequence of configurations $c_0, c_1, c_2, \dots$
 $c_0 = (0, r_0, \dots, r_n)$ is **initial configuration**
each c_{i+1} determined from $c_i = (r_i, r_0, \dots, r_n)$ by performing **inst.** labelled L_i

Halting Configuration $c_m = (r, r_0, \dots, r_n)$
inst. labelled L_i is **either** $HALT$ i.e. **Proper Halt** or performs jump to **nonexistent label** i.e. **Erroneous Halt**
e.g. $L_i: R_1^+ \rightarrow L_2$ $L_2: HALT$ $\Rightarrow$ **halts erroneously**

Non-Halting Computation $\Rightarrow$ infinite sequence
 $c_0, c_1, \dots$ e.g. $L_0: R_1^+ \rightarrow L_0$ $L_1: HALT$

RM Graphical Representation
one node per **inst.** $L_i: B_i$
each node labelled by $[L_i]$ where $[L_i]$ is register of **inst. body** B_i

Arcs are jumps: **initial instruction** is **START**

Instruction Representation
 $R^+ \rightarrow L^+ \Rightarrow R^+ \rightarrow [L]$
 $R^- \rightarrow L^+ \Rightarrow R^- \rightarrow [L]$
 $HALT \Rightarrow HALT$
 $L_0 \Rightarrow START \rightarrow [L_0]$

$f: N^0 \rightarrow N$ is **(RM) Computable Function** if:
There is RM M with $\geq n+1$ registers, such that for all $(x_1, \dots, x_n) \in N^n, y \in N$ we have **initial configuration** $c_0 = (0, x_1, \dots, x_n, 0, \dots)$ and M halts with $R_0 = y$ **iff** $f(x_1, \dots, x_n) = y$
Basic computable functions $\Rightarrow$ **addition**, **multiplication**, **projection** $(x, y) \mapsto x$; **constant** $x \mapsto b$; **truncated subtraction** $(x, y) \mapsto \max(0, x-y)$; **integer division** $(x, y) \mapsto \lfloor \frac{x}{y} \rfloor$ $y > 0$; **integer remainder** $(x, y) \mapsto x - y \cdot \lfloor \frac{x}{y} \rfloor$ $y > 0$

$(x, y) \mapsto \begin{cases} x-y & x \geq y \\ 0 & x < y \end{cases}$ $y > 0$

exp. and log. $\text{base-2 } x \mapsto 2^x, x \mapsto \lfloor \log_2(x) \rfloor$ $x > 0$
 $x = 0 \Rightarrow 0$

Numerical Coding

Numerical Coding of Pairs $\Rightarrow$ for $\alpha, \beta \in N$ define:
 $(\alpha, \beta) \triangleq 2^{\alpha}(2\beta+1)$ is a **bijection** between N^2 and N^+
where $N^+ = \{n \in N | n > 0\}$

Binary form $\Rightarrow 0b(\alpha, \beta) = 0b \left[\begin{matrix} \alpha & \beta \end{matrix} \right]$
 $(\alpha, \beta) \in \{0, 1\}^2 \Rightarrow 1+2^{\alpha}(2\beta+1)-1$
 $(-, -): N^2 \rightarrow N$ is a **bijection** between N^2 and N
Binary form $\Rightarrow 0b(\alpha, \beta) = 0b \left[\begin{matrix} \alpha & \beta \end{matrix} \right]$
EXAMPLE: $27 = 0b11011 = (0, 13) = (2, 3)$

Numerical Coding of Lists:
 $List(N)$ is set of all finite lists of **natural numbers**:
 $Empty List \Rightarrow [] \in List(N)$
 $List Cons \Rightarrow x::l \in List(N) \iff x \in N \wedge l \in List(N)$
Notation $\Rightarrow x::(x_1, x_2, \dots, x_n) \triangleq x_1::x_2::\dots::x_n::[]$

Define **bijection** $(-, -): List(N) \rightarrow N$ inductively:
 $[] \mapsto 0$; $x::l \mapsto 2^x(1 + 2^{b(l)})$
Binary form $\Rightarrow$

$0b[x_1, \dots, x_n, 1, x_n] = 0b \left[\begin{matrix} 1 & 0 & \dots & 0 & 1 \\ x_n & & & & \end{matrix} \right]$
Let $P = [L_0: B_0, L_1: B_1, \dots, L_n: B_n]$ be an **RM program**:
Numerical Coding of Bodies (B) is defined by

$[R_i^+ \rightarrow L_j^+ \rightarrow L_k^+] \triangleq (2i, j)$
 $[R_i^- \rightarrow L_j^+ \rightarrow L_k^+] \triangleq (2i+1, (j, k))$ and $[HALT]$ defines a **bijection**
 $x=0 \Rightarrow body(x) = HALT$
 $x=1 \Rightarrow body(x) = x$
 $y=2i \Rightarrow body(x) = R_i^+ \rightarrow L_j^+$
 $y=2i+1 \Rightarrow body(x) = R_i^- \rightarrow L_j^+ \rightarrow L_k^+$
Any $e \in N$ decodes to **unique program** $prog(e)$ called **program with index e**

Register Machine Gadgets

RM Gadgets $\Rightarrow$ **partial RM graph** with **1 entry wire** & ≥ 1 **more exit wires**
Input/output registers specified in **gadget's name**
 $R_i = x$ [in config. c] means $c = (r, r_0, \dots, r_n)$ with $r_i = x$
initial configuration $\Rightarrow c_0 = (0, r_0, \dots, r_n)$

these **scratch registers** are **initially set to zero** & **upon exit must return to zero**
Compose **smaller gadgets** to create **bigger gadgets** $\Rightarrow$ can **rename** clashing scratch register names

Common RM Gadgets:

"zero R_0 "
entry $\Rightarrow$ **"add R_1 to R_0 "**
exit $\Rightarrow$ **"copy R_1 to R_2 "**

"copy R_1 to R_2 " **"copy R_1 to R_2 and R_3 "**
entry $\Rightarrow$ **"zero R_2 "** **"add R_1 to R_2 "**
exit $\Rightarrow$ **"copy R_1 to R_3 "** **"copy R_1 to R_2 "**

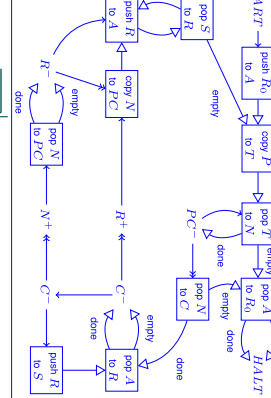
"multiply R_1 by R_2 to R_0 "
entry $\Rightarrow$ **"zero R_0 "** **"add R_2 to R_0 "**
exit $\Rightarrow$ **"copy R_1 to R_2 "**

Gadget: "push X to L "
entry $\Rightarrow$ **"zero R_0 "** **"add R_2 to R_0 "**
exit $\Rightarrow$ **"copy R_1 to R_2 "**

Halting Problem for Register Machines
Register machine H decides the Halting Problem if:
For all $e, a_1, \dots, a_n \in N^n$ starting H with $R_0 = 0$, $R_i = e_i$ $R_2 = [a_1, \dots, a_n]$ and **all other registers zeroed**
the computation of H halts with R_0 containing 0 or 1 **iff** $f(a_1, \dots, a_n) = 1$
and when the computation halts, $R_0 = 1$ **iff** $f(a_1, \dots, a_n) = 1$
the register machine program with index e eventually halts
when started with $R_0 = 0, R_1 = a_1, \dots, R_n = a_n$ and **all other registers zeroed**
THEOREM: No such register machine H can exist

Gadget: "pop L to X "
entry $\Rightarrow$ **"zero R_0 "** **"add R_2 to R_0 "**
exit $\Rightarrow$ **"copy R_1 to R_2 "**

NOTE: the **Hollow Arrow** coming out of **gadgets** represents **all outputs** combined
Universal Register Machine (URM)



Substitution $\text{sub } x \leftarrow (-) \lambda m.n$. n pred m
 $\{(-) m\}$ yields m when m and n and 0 otherwise

Pairs and Linked Lists
 We can use **Viero** as our **pair constructor**
 $\text{pair} = \lambda V \lambda x y.zxy$
 $(\lambda x y) = \text{pair } x y$ [represents the pair (x, y)]
 $\text{fst} = \lambda p.p \text{ K}$ $\text{snd} = \lambda p.p \text{ x}$ $(\lambda xy.x)$
 $\text{snd} (\lambda xy.y) = \text{K}$ $\text{fst} (\lambda xy.y) = \text{K}$
 the **empty list** element $\text{nil} = \lambda x.x \text{ K}$ and
 a check for it is $\text{isNil} = \lambda x.x (\lambda yz.\text{K})$
 Create **linked lists** as nested **pairs**
 i.e. $[x_1, \dots, x_n]$ encoded as $(x_1, (\dots, (x_n, \text{nil})))$

Pairs and Linked Lists

We can use `Vec` as our **pair constructor**

i.e. `pair = V => λx.yz.xzy`

`(V x y) pair x y` represents the pair `(x, y)`

`fst = λp.p K = λp.p (λ(x.y).x)`

`snd = λp.p KI = λp.p (λ(x.y).y)`

the **empty list** element `nil = λx.K` and
 a check for it is `isNil = λx.x (λ(yz.K))`

Create **linked lists** as nested **pairs**

i.e. `[x1, ..., xn]` encoded as `{x1, (... , {xn, nil})}`